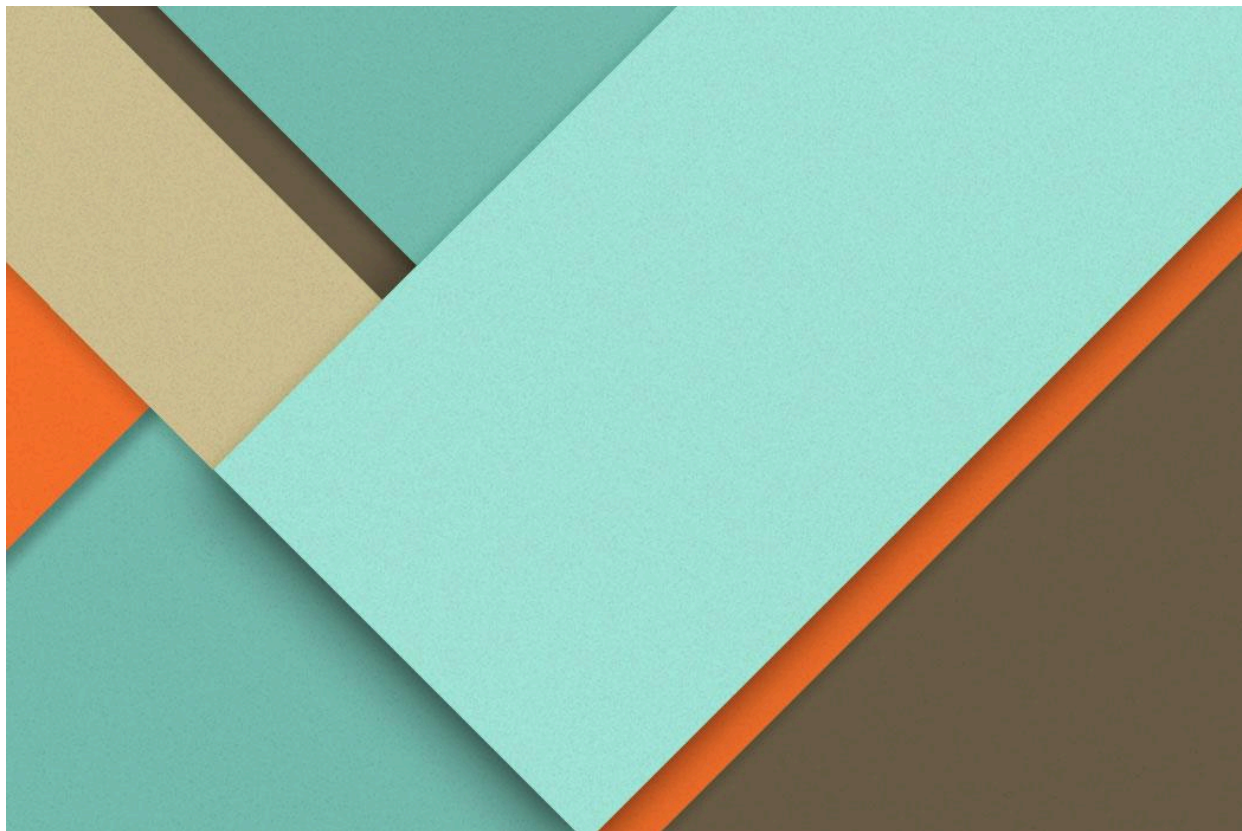




UD5: Manual Técnico Hospital

05/05/2026

José Manuel Sánchez Rosal

2º DAM

IES Antonio Gala

1400 Palma del Río (Córdoba)

Índice

Índice.....	1
1. Arquitectura de Modelos y Base de Datos (ORM).....	1
1.1. Modelo: HospitalEspecialidad.....	2
1.2. Modelo: HospitalMedico.....	3
1.3. Modelo: HospitalCita.....	5
2. Diseño de Vistas e Interfaz (XML).....	6
2.1. Especialidad.....	7
2.1.1. Especialidad-Search (mejora).....	7
2.1.2. Especialidad-Lista (V2).....	7
2.1.3. Especialidad-Kanban (V2).....	8
2.1.4. Especialidad-Formulario (V2).....	10
2.2. Médico.....	12
2.2.1. Médico-Search (nueva).....	12
2.2.2. Médico-Lista (V2).....	12
2.2.3. Médico-Kanban (V2).....	13
2.2.4. Médico-Form (V2).....	15
2.3. Cita.....	16
2.3.1. Cita-Search (nueva).....	16
2.3.2. Cita-Lista (V2).....	17
2.3.3. Cita-Form (V2).....	18
2.4. Ventanas de Acción.....	19
2.5. Menús.....	20
3. Gestión de Permisos y Accesos.....	21
3.1. Definición de Roles (security.xml).....	21
3.2. Asignación de Derechos (ir.model.access.csv).....	22
4. Declaración del Módulo.....	23
4. Documentación y Estándares.....	23

1. Arquitectura de Modelos y Base de Datos (ORM)

El núcleo del módulo hospitalario se ha desarrollado utilizando el ORM (Object-Relational Mapping) de Odoo, mapeando las clases de Python directamente a tablas de PostgreSQL. Se han aplicado estándares de documentación Google Docstrings para facilitar el mantenimiento y la lectura del código fuente por parte de futuros desarrolladores.

1.1. Modelo: HospitalEspecialidad

Este modelo actúa como tabla maestra para categorizar a los facultativos y gestionar la logística física (consultas) del hospital:

```
# -*- coding: utf-8 -*-
"""
Módulo de Gestión Hospitalaria.

Define la arquitectura base de datos para el sistema ERP-CRM del
hospital.
Proporciona los modelos ORM para Especialidades, Médicos y Citas,
estableciendo
las relaciones relacionales necesarias para la integridad de los datos.
"""

from odoo import models, fields

class HospitalEspecialidad(models.Model):
    """
    Modelo que representa las especialidades médicas del hospital.

    Permite categorizar a los facultativos y asignarles un número de
    consulta
    físico dentro de las instalaciones del centro.

    :ivar name: Nombre identificativo de la especialidad.
    :vartype name: fields.Char
```

```
:ivar consulta: Sala asignada para el desarrollo de la especialidad.
:var type consulta: fields.Selection
    :ivar medico_ids: Relación One2many para extraer los médicos que
    pertenecen a esta especialidad.
:var type medico_ids: fields.One2many
"""
_name = 'hospital.especialidad'
_description = 'Especialidades del Hospital'

name = fields.Char(string='Nombre', required=True, size=40)
consulta = fields.Selection([
    ('consulta1', 'Consulta 1'),
    ('consulta2', 'Consulta 2'),
    ('consulta3', 'Consulta 3'),
    ('consulta4', 'Consulta 4'),
    ('consulta5', 'Consulta 5'),
], string='Consultas', default='consulta1')

medico_ids = fields.One2many(
    comodel_name='hospital.medico',
    inverse_name='especialidad_id',
    string='Médicos en esta especialidad'
)
```

1.2. Modelo: HospitalMedico

Representa la entidad central del módulo. Implementa restricciones de tamaño en los campos de texto, validaciones de tipo fecha y campos binarios para el tratamiento de imágenes (`fields.Image`). Incorpora relaciones `Many2one` para heredar la especialidad y `One2many` para visualizar de forma inversa las citas programadas:

```
class HospitalMedico(models.Model):
    """
    Modelo que gestiona el personal facultativo del hospital.
```

Almacena los datos de contacto y perfil profesional de cada médico.

Actúa como tabla intermedia lógica entre las especialidades y las citas.

```
:ivar name: Nombre y apellidos completos del facultativo.
:vartype name: fields.Char
:ivar domicilio: Dirección postal de residencia.
:vartype domicilio: fields.Char
:ivar telefono: Número de teléfono de contacto (limitado a 9
dígitos).
:vartype telefono: fields.Char
:ivar fecha_ingreso: Fecha de contratación en el sistema
hospitalario.
:vartype fecha_ingreso: fields.Date
:ivar image: Fotografía de perfil utilizada en las vistas Kanban y
List.
:vartype image: fields.Image
:ivar especialidad_id: Relación Many2one que vincula al médico con su
especialidad.
:vartype especialidad_id: fields.Many2one
:ivar cita_ids: Relación One2many para extraer el historial de citas
programadas del médico.
:vartype cita_ids: fields.One2many
"""
_name = 'hospital.medico'
_description = 'Médicos del Hospital'
name = fields.Char(string='Nombre y Apellidos', required=True,
size=40)
domicilio = fields.Char(string='Domicilio', required=True, size=40)
telefono = fields.Char(string='Teléfono', required=True, size=9)
fecha_ingreso = fields.Date(string='Fecha de Ingreso', required=True)
image = fields.Image(string="Foto de Perfil")

especialidad_id = fields.Many2one(
    comodel_name='hospital.especialidad',
```

```
        string='Especialidad',
        required=True
    )
    cita_ids = fields.One2many(
        comodel_name='hospital.cita',
        inverse_name='medico_id',
        string='Citas Asignadas'
    )
```

1.3. Modelo: HospitalCita

Modelo transaccional orientado al registro de eventos temporales. Utiliza campos de tipo **Datetime** para asegurar la precisión horaria y cruza la información entre las entidades independientes de pacientes y el personal médico:

```
class HospitalCita(models.Model):
    """
        Modelo transaccional para la programación y registro de citas
        médicas.

        Vincula un paciente con un facultativo en una marca de tiempo
        específica.

        Implementa campos relacionados para optimizar la normalización de la
        base de datos.

        :ivar paciente: Nombre y apellidos del paciente que solicita
        asistencia.
        :vartype paciente: fields.Char
        :ivar medico_id: Relación Many2one con el facultativo asignado a la
        consulta.
        :vartype medico_id: fields.Many2one
        :ivar fecha_cita: Marca de tiempo exacta programada para la visita
        médica.
        :vartype fecha_cita: fields.Datetime
        :ivar especialidad_id: Campo relacionado (Read-Only) que hereda
        automáticamente
```

```
la especialidad del medico_id seleccionado.
:vartype especialidad_id: fields.Many2one
"""
_name = 'hospital.cita'
_description = 'Citas de Pacientes'

paciente = fields.Char(string='Paciente (Nombre y Apellido)',
required=True, size=40)

medico_id = fields.Many2one(
    comodel_name='hospital.medico',
    string='Médico',
    required=True
)

fecha_cita = fields.Datetime(string='Fecha y hora de la cita',
required=True)

especialidad_id = fields.Many2one(
    comodel_name='hospital.especialidad',
    related='medico_id.especialidad_id',
    string='Especialidad',
    store=True,
    readonly=True
)
```

2. Diseño de Vistas e Interfaz (XML)

Las interfaces de usuario se han definido de manera declarativa mediante XML. Se ha adaptado el código a la sintaxis estricta de Odoo 18, sustituyendo las etiquetas obsoletas `<tree>` por `<list>`. Se han implementado a posteriori, mejoras en todas las vistas usando **Bootstrap**. El desarrollo incluye la implementación de bloques `<kanban>` avanzados que hacen uso de directivas QWeb (`t-att-src`) para renderizar dinámicamente datos binarios almacenados en base de datos:

2.1. Especialidad

2.1.1. Especialidad-Search (mejora)

- Se incorpora una vista de búsqueda personalizada. Además de permitir la búsqueda por nombre y número de consulta, se añade un filtro de agrupación (**group_by**) que permite categorizar la vista lista basándose en la consulta asignada:

```
<odoo>
<record id="view_hospital_especialidad_search" model="ir.ui.view">
  <field name="name">hospital.especialidad.search</field>
  <field name="model">hospital.especialidad</field>
  <field name="arch" type="xml">
    <search>
      <field name="name"/>
      <field name="consulta"/>
      <group expand="0" string="Agrupar por">
        <filter string="Consulta" name="group_by_consulta"
context="{ 'group_by': 'consulta' }"/>
      </group>
    </search>
  </field>
</record>
```

2.1.2 Especialidad-Lista (V2)

- Se aplican decoradores visuales a la etiqueta **<list>**. El nombre de la especialidad se renderiza en negrita (**decoration-bf**) y el campo consulta se estiliza mediante un componente visual tipo etiqueta (**widget="badge"**):

```
<record id="view_hospital_especialidad_list" model="ir.ui.view">
  <field name="name">hospital.especialidad.list</field>
  <field name="model">hospital.especialidad</field>
  <field name="arch" type="xml">
    <list decoration-info="consulta">
      <field name="name" string="Especialidad"
```



```

decoration-bf="1"/>
        <field name="consulta" optional="show" widget="badge"
decoration-info="1"/>
    </list>
</field>
</record>

```

2.1.3. Especialidad-Kanban (V2)

- Se ha rediseñado completamente la vista mediante el uso de clases de Bootstrap (**d-flex**, **text-center**, **p-4**) para lograr un diseño responsivo. Se implementa lógica condicional QWeb (**<t t-if>**) para renderizar iconos de FontAwesome de diferentes colores según la especialidad médica leída de la base de datos:

```

<record id="view_hospital_especialidad_kanban" model="ir.ui.view">
    <field name="name">hospital.especialidad.kanban</field>
    <field name="model" >hospital.especialidad</field>
    <field name="arch" type="xml">
        <kanban class="o_kanban_mobile" sample="1">
            <field name="name"/>
            <templates>
                <t t-name="kanban-box">
                    <div t-attf-class="oe_kanban_global_click
oe_kanban_card p-4 d-flex flex-column justify-content-center text-center
position-relative" style="min-height: 200px;">
                        <div class="position-absolute top-0 end-0
p-3">
                            <t t-if="record.name.raw_value ==
'Cardiologia' or record.name.raw_value == 'Cardiología'">
                                <i class="fa fa-heartbeat fa-4x
text-danger"/>
                            </t>
                            <t t-elif="record.name.raw_value ==
'Nutricion' or record.name.raw_value == 'Nutrición'">

```

```

                                <i class="fa fa-leaf fa-4x
text-success"/>

                                </t>

                                <t t-elif="record.name.raw_value ==
'Aparato Locomotor' or record.name.raw_value == 'Aparato locomotor'">
                                <i class="fa fa-wheelchair fa-4x
text-warning"/>

                                </t>

                                <t t-elif="record.name.raw_value ==
'Psicologia' or record.name.raw_value == 'Psicología'">
                                <i class="fa fa-puzzle-piece fa-4x
text-info"/>

                                </t>
                                <t t-else="">
                                <i class="fa fa-user-md fa-4x
text-primary"/>

                                </t>
</div>
<div class="o_kanban_record_top mb-3">
    <div class="o_kanban_record_headings">
        <strong class="o_kanban_record_title
fs-3 text-dark mt-3">

                                <field name="name"/>
                                </strong>
        </div>
    </div>
    <div class="o_kanban_record_bottom mt-auto">
        <div class="oe_kanban_bottom_center">
            <span class="badge rounded-pill
text-bg-info fs-5 px-3 py-2">

                                <i class="fa fa-door-open me-2"/>
                                <field name="consulta"/>
            </span>
        </div>
    </div>
</div>
</div>

```

```

        </t>
    </templates>
</kanban>
</field>
</record>

```

2.1.4. Especialidad-Formulario (V2)

- Se estructura la cabecera mediante la clase `oe_title`. El selector de consultas se mejora a nivel de usabilidad transformándolo en un conjunto de botones de radio horizontales (`widget="radio" options="{ 'horizontal': true }"`):

```

<record id="view_hospital_especialidad_form" model="ir.ui.view">
    <field name="name">hospital.especialidad.form</field>
    <field name="model">hospital.especialidad</field>
    <field name="arch" type="xml">
        <form>
            <sheet>
                <div class="oe_avatar text-center">
                    <i class="fa fa-heartbeat fa-5x text-danger"
invisible="name != 'Cardiologia' and name != 'Cardiología'"/>
                    <i class="fa fa-leaf fa-5x text-success"
invisible="name != 'Nutricion' and name != 'Nutrición'"/>
                    <i class="fa fa-wheelchair fa-5x text-warning"
invisible="name != 'Aparato Locomotor' and name != 'Aparato
locomotor'"/>
                    <i class="fa fa-puzzle-piece fa-5x text-info"
invisible="name != 'Psicologia' and name != 'Psicología'"/>
                    <i class="fa fa-user-md fa-5x text-primary"
invisible="name == 'Cardiologia' or name == 'Cardiología' or name ==
'Nutricion' or name == 'Nutrición' or name == 'Aparato Locomotor' or
name == 'Aparato locomotor' or name == 'Psicologia' or name ==
'Psicología'"/>
                </div>
            </sheet>
        </form>
    </field>
</record>

```

```
<div class="oe_title">
    <label for="name" class="oe_edit_only"
string="Nombre de Especialidad"/>
    <h1>
        <field name="name" placeholder="Ej.
Cardiología"/>
    </h1>
</div>
<group>
    <group>
        <field name="consulta" widget="radio"
options="{ 'horizontal': true }"/>
    </group>
</group>

<notebook>
    <page string="Médicos Asignados">
        <field name="medico_ids">
            <list>
                <field name="name"/>
                <field name="telefono"
widget="phone"/>
            </list>
        </field>
    </page>
</notebook>
</sheet>
</form>
</field>
</record>
```

2.2. Médico

2.2.1. Médico-Search (nueva)

- Se amplía el dominio de búsqueda nativo. El atributo `filter_domain` permite que al escribir en la barra superior se busque simultáneamente tanto en el nombre del facultativo como en su número de teléfono. Se añade agrupación por especialidad:

```
<record id="view_hospital_medico_search" model="ir.ui.view">
  <field name="name">hospital.medico.search</field>
  <field name="model">hospital.medico</field>
  <field name="arch" type="xml">
    <search>
      <field name="name" filter_domain="['|', ('name', 'ilike',
self), ('telefono', 'ilike', self)]"/>
      <field name="especialidad_id"/>
      <group expand="0" string="Agrupar por">
        <filter string="Especialidad"
name="group_by_especialidad" context="{ 'group_by': 'especialidad_id' }"/>
      </group>
    </search>
  </field>
</record>
```

2.2.2. Médico-Lista (V2)

- La tabla se enriquece incrustando la fotografía del médico mediante el widget `image` con dimensiones fijadas como avatar (`oe_avatar`). Se incluyen widgets específicos para formatear el teléfono (`phone`) y etiquetas de color (`decoration-primary`) para la especialidad:

```
<record id="view_hospital_medico_list" model="ir.ui.view">
  <field name="name">hospital.medico.list</field>
  <field name="model">hospital.medico</field>
```

```

<field name="arch" type="xml">
  <list>
    <field name="image" widget="image" class="oe_avatar"
options="{ 'size': [35, 35]}"/>
    <field name="name" decoration-bf="1"/>
    <field name="especialidad_id" widget="badge"
decoration-primary="1"/>
    <field name="telefono" widget="phone"/>
    <field name="fecha_ingreso" optional="hide"/>
  </list>
</field>
</record>

```

2.2.3. Médico-Kanban (V2)

- Se utiliza la clase estructural avanzada `o_kanban_record_has_image_fill` combinada con `o_kanban_image_fill_left`. Esto permite que la imagen binaria del médico ocupe proporcionalmente todo el margen izquierdo de la tarjeta Kanban. Se añaden iconos informativos para los datos de contacto:

```

<record id="view_hospital_medico_kanban" model="ir.ui.view">
  <field name="name">hospital.medico.kanban</field>
  <field name="model">hospital.medico</field>
  <field name="arch" type="xml">
    <kanban sample="1">
      <field name="id"/>
      <field name="image"/>
      <field name="name"/>
      <field name="especialidad_id"/>
      <field name="telefono"/>
      <templates>
        <t t-name="kanban-box">
          <div class="oe_kanban_global_click
o_kanban_record_has_image_fill p-2" style="min-height: 180px;">

```

```

        <t t-if="record.image.raw_value">
            <div class="o_kanban_image_fill_left
d-none d-md-block"
t-attf-style="background-image:url('#{kanban_image('hospital.medico',
'image', record.id.raw_value)}'); width: 140px;"/>
        </t>
        <t t-else="">
            <div class="o_kanban_image" style="width:
140px;"/>
        </t>

        <div class="oe_kanban_details d-flex
flex-column justify-content-between p-3">
            <div>
                <strong class="o_kanban_record_title
fs-3 text-dark"><field name="name"/></strong>
                <div class="mt-2">
                    <span class="badge
text-bg-primary fs-6 px-3 py-2"><field name="especialidad_id"/></span>
                </div>
            </div>
            <div class="o_kanban_record_bottom mt-3">
                <div class="oe_kanban_bottom_left
text-muted fs-5">
                    <i class="fa fa-phone me-2
text-info"/> <field name="telefono" widget="phone"/>
                </div>
            </div>
        </div>
    </div>
</t>
</templates>
</kanban>
</field>
</record>

```

2.2.4. Médico-Form (V2)

- La vista de edición integra un previsualizador de imagen ([preview_image](#)). Los campos de datos se distribuyen ordenadamente utilizando etiquetas `<group>` anidadas, e implementan los widgets [selection](#), [phone](#) y [date](#) para asegurar la correcta entrada de datos:

```
<record id="view_hospital_medico_form" model="ir.ui.view">
    <field name="name">hospital.medico.form</field>
    <field name="model">hospital.medico</field>
    <field name="arch" type="xml">
        <form>
            <sheet>
                <field name="image" widget="image" class="oe_avatar"
options="{ 'preview_image': 'image' }" />
                <div class="oe_title">
                    <label for="name" class="oe_edit_only"
string="Nombre y Apellidos"/>
                    <h1>
                        <field name="name" placeholder="Dr. / Dra." />
                    </h1>
                </div>
                <group>
                    <group>
                        <field name="especialidad_id"
widget="selection"/>
                    <group>
                        <field name="telefono" widget="phone"/>
                    </group>
                </group>
                <group>
                    <field name="domicilio"/>
                    <field name="fecha_ingreso" widget="date"/>
                </group>
            </group>
        </form>
    </field>
</record>
```



```

        <notebook>
            <page string="Citas de Pacientes">
                <field name="cita_ids">
                    <list default_order="fecha_cita desc"
decoration-success="fecha_cita          &gt;=          current_date"
decoration-danger="fecha_cita &lt; current_date">
                        <field name="fecha_cita"
widget="datetime"/>
                    <field name="paciente"/>
                </list>
            </field>
        </page>
    </notebook>
</sheet>
</form>
</field>
</record>

```

2.3. Cita

2.3.1. Cita-Search (nueva)

- Se introduce un dominio dinámico (`context_today().strftime('%Y-%m-%d')`) en los filtros para aislar con un solo clic las 'Citas Futuras'. Además, permite agrupar los registros por Médico o por el día exacto de la cita:

```

<record id="view_hospital_cita_search" model="ir.ui.view">
    <field name="name">hospital.cita.search</field>
    <field name="model">hospital.cita</field>
    <field name="arch" type="xml">
        <search>
            <field name="paciente"/>
            <field name="medico_id"/>
            <filter string="Citas Futuras" name="citas_futuras"

```

```

domain="(['fecha_cita',
context_today().strftime('%Y-%m-%d'))]"/>
    <group expand="0" string="Agrupar por">
        <filter string="Médico" name="group_by_medico"
context="{ 'group_by': 'medico_id' }"/>
        <filter string="Fecha" name="group_by_fecha"
context="{ 'group_by': 'fecha_cita:day' }"/>
    </group>
</search>
</field>
</record>

```

2.3.2. Cita-Lista (V2)

- Se fuerza la ordenación descendente por defecto (`default_order="fecha_cita desc"`). Se implementa un sistema de semaforización mediante decoradores condicionales a nivel de lista: las citas programadas en fecha actual o futura se resaltan en verde (`decoration-success`), mientras que las vencidas se marcan en rojo (`decoration-danger`):

```

<record id="view_hospital_cita_list" model="ir.ui.view">
    <field name="name">hospital.cita.list</field>
    <field name="model">hospital.cita</field>
    <field name="arch" type="xml">
        <list default_order="fecha_cita desc"
decoration-success="fecha_cita &gt;= current_date"
decoration-danger="fecha_cita &lt; current_date">
            <field name="fecha_cita" widget="datetime"/>
            <field name="paciente" decoration-bf="1"/>
            <field name="medico_id" widget="many2one_avatar"/>
            <field name="especialidad_id" widget="badge"
decoration-info="1"/>
        </list>
    </field>
</record>

```

2.3.3. Cita-Form (V2)

- Destaca la inclusión del componente visual `<widget name="web_ribbon">` que simula una cinta de estado en la esquina del formulario. Para la selección del médico, se utiliza `many2one_avatar`, mostrando la foto del facultativo directamente en el desplegable de asignación:

```
<record id="view_hospital_cita_form" model="ir.ui.view">
  <field name="name">hospital.cita.form</field>
  <field name="model">hospital.cita</field>
  <field name="arch" type="xml">
    <form>
      <sheet>
        <widget name="web_ribbon" title="Programada"
bg_color="text-bg-success"/>
        <div class="oe_title">
          <label for="paciente" class="oe_edit_only"
string="Paciente (Nombre y Apellido)"/>
          <h1>
            <field name="paciente" placeholder="Nombre
completo"/>
          </h1>
        </div>
        <group>
          <group>
            <field name="medico_id"
widget="many2one_avatar"/>
            <field name="especialidad_id"/>
          </group>
          <group>
            <field name="fecha_cita" widget="datetime"/>
          </group>
        </group>
      </sheet>
    </form>
  </field>
</record>
```

2.4. Ventanas de Acción

- Las ventanas de acción (`ir.actions.act_window`) actúan como el motor que vincula los modelos de datos con la interfaz de usuario. En este módulo, se han configurado cuidadosamente para definir el comportamiento de apertura de cada entidad. Se utiliza el atributo `view_mode` para establecer la jerarquía de visualización, priorizando la vista **Kanban** en Médicos y Especialidades por su alto valor gráfico, mientras que en Citas se prioriza la vista **Lista** para facilitar la gestión de la agenda. Además, se definen mensajes de ayuda (`help`) para guiar al usuario en caso de que no existan registros creados:

```
<record id="action_hospital_especialidad" model="ir.actions.act_window">
    <field name="name">Especialidades</field>
    <field name="res_model">hospital.especialidad</field>
    <field name="view_mode">kanban,list,form</field>
    <field name="help" type="html">
        <p class="o_view_nocontent_smiling_face">
            Crea la primera especialidad del hospital
        </p>
    </field>
</record>

<record id="action_hospital_medico" model="ir.actions.act_window">
    <field name="name">Médicos</field>
    <field name="res_model">hospital.medico</field>
    <field name="view_mode">kanban,list,form</field>
    <field name="help" type="html">
        <p class="o_view_nocontent_smiling_face">
            Registra un nuevo médico
        </p>
    </field>
</record>

<record id="action_hospital_cita" model="ir.actions.act_window">
    <field name="name">Citas</field>
    <field name="res_model">hospital.cita</field>
```

```
<field name="view_mode">list,form</field>
<field name="help" type="html">
  <p class="o_view_nocontent_smiling_face">
    Programa una nueva cita
  </p>
</field>
</record>
```

2.5. Menús

- La arquitectura de menús define la navegación del sistema de forma jerárquica y organizada. Se ha implementado un menú raíz principal que actúa como contenedor en la barra superior de Odoo. A partir de este, se despliegan tres submenús vinculados a sus respectivas ventanas de acción. Se hace uso del atributo **sequence** para garantizar un orden lógico de trabajo: primero la configuración de la estructura (**Especialidades**), segundo el alta del personal (**Médicos**) y, finalmente, la operativa diaria (**Citas**). Esta disposición asegura que el flujo de trabajo sea intuitivo y profesional para el personal del centro:

```
<menuitem id="menu_hospital_root" name="Hospital"/>
  <menuitem id="menu_hospital_especialidades" name="Especialidades"
parent="menu_hospital_root"          action="action_hospital_especialidad"
sequence="10"/>
    <menuitem id="menu_hospital_medicos" name="Médicos"
parent="menu_hospital_root"          action="action_hospital_medico"
sequence="20"/>
      <menuitem id="menu_hospital_citas" name="Citas"
parent="menu_hospital_root"          action="action_hospital_cita"
sequence="30"/>
</odoo>
```

3. Gestión de Permisos y Accesos

La seguridad del módulo no se basa en los grupos por defecto, sino que se ha implementado un sistema de roles jerárquicos a medida. La arquitectura de seguridad se divide en dos ficheros clave:

3.1. Definición de Roles (`security.xml`)

Se ha creado una categoría específica para el módulo y dos grupos de acceso (`res.groups`): un rol base para Recepcionistas y un rol heredado para Administradores, aplicando el principio de herencia de permisos (`implied_ids`):

```
<?xml version="1.0" encoding="utf-8"?>
<odoo>
  <data noupdate="1">
    <record id="module_category_hospital" model="ir.module.category">
      <field name="name">Gestión Hospitalaria</field>
      <field name="description">Nivel de acceso para el sistema del
hospital</field>
      <field name="sequence">10</field>
    </record>

    <record id="group_hospital_user" model="res.groups">
      <field name="name">Usuario / Recepcionista</field>
      <field name="category_id" ref="module_category_hospital"/>
      <field name="implied_ids" eval="[(4,
ref('base.group_user'))]" />
      <field name="comment">Este usuario puede gestionar citas y
consultar médicos/especialidades.</field>
    </record>

    <record id="group_hospital_manager" model="res.groups">
      <field name="name">Administrador / Responsable</field>
      <field name="category_id" ref="module_category_hospital"/>
      <field name="implied_ids" eval="[(4,
```

```
ref('group_hospital_user')))]"/>
    <field name="comment">Acceso total para configurar el centro
(especialidades y médicos).</field>
</record>
</data>
</odoo>
```

3.2. Asignación de Derechos ([ir.model.access.csv](#))

Las Listas de Control de Acceso (ACL) se han mapeado explícitamente a estos nuevos grupos. El personal base solo tiene permisos completos sobre las citas, mientras que los administradores poseen control total (CRUD = 1,1,1,1) sobre la estructura del hospital:

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
access_hospital_especialidad_mgr,hospital.especialidad,model_hospital_especialidad,group_hospital_manager,1,1,1,1
access_hospital_medico_mgr,hospital.medico,model_hospital_medico,group_hospital_manager,1,1,1,1
access_hospital_cita_mgr,hospital.cita,model_hospital_cita,group_hospital_manager,1,1,1,1
access_hospital_especialidad_user,hospital.especialidad,model_hospital_especialidad,group_hospital_user,1,0,0,0
access_hospital_medico_user,hospital.medico,model_hospital_medico,group_hospital_user,1,0,0,0
access_hospital_cita_user,hospital.cita,model_hospital_cita,group_hospital_user,1,1,1,0
```

4. Declaración del Módulo

El punto de entrada del módulo define sus metadatos, dependencias y, de manera crítica, el orden de carga de los archivos de datos. Es imperativo que el archivo de seguridad ([security.xml](#)) y el control de accesos ([ir.model.access.csv](#)) se carguen antes que las vistas para evitar errores de despliegue en el servidor:

```
{
    'name': 'hospital',
    'version': '1.0',
    'summary': 'Módulo para la gestión de especialidades, médicos y citas',
    'description': 'Permite gestionar el registro de pacientes, asignación de médicos y programación de citas.',
    'author': 'José Manuel Sánchez Rosal',
    'depends': ['base'],
    'data': [
        'views/reports.xml',
        'security/security.xml',
        'security/ir.model.access.csv',
        'views/views.xml',
    ],
    'installable': True,
    'application': True,
    'icon': '/hospital/static/description/icon.png',
}
```

4. Documentación y Estándares

Todo el código Python ha sido documentado mediante la extensión [autoDocstring](#), garantizando que cada atributo y clase contenga su descripción técnica integrada. Los nombres de variables, las relaciones y los decoradores cumplen con las directrices PEP8 y los estándares de desarrollo oficiales de Odoo.